Mongo Cheatsheet

Inserarea

```
db.collection.insert(
    <document or array of documents>,
    {
      writeConcern: <document>,
      ordered: <boolean>
    }
)
```

Pentru a insera documentul în colecția students:

```
db.students.insert(
{
    "student":
        {
            "firstname": "Ion",
            "lastname": "Popescu"
        },
    "an": 4,
    "grupa": "342C3",
    "materii":
        [
            {"nume": "Comp", "an": 4},
            {"nume": "BD2" , "an": 4},
            {"nume": "SO2" , "an": 4}
        ],
    "cunostinte": ["SQL", "Java", "PL/SQL"]
}
)
```

Pentru a vă ușura munca puteți să vă creați variabile la nivelul consolei. Dacă se dorește inserarea mai multor înregistrări, se poate declara un vector.

```
var stud = {"student": { "firstname": "Andrei", "lastname": "Ionescu"}, "an": 4, "grupa": "341C4",

db.students.insert(stud)

var studs = [
    {"student": { "firstname": "George", "lastname": "Popescu"}, "an": 4, "grupa": "341C2", "mater
    {"student": { "firstname": "Georgiana", "lastname": "Petrescu"}, "an": 4, "grupa": "341C2", "m
    {"student": { "firstname": "Valentina", "lastname": "Vasilescu"}, "an": 3, "grupa": "331CA", "
    {"student": { "firstname": "Grigore", "lastname": "Ionescu"}, "an": 4, "grupa": "342C2", "mate
    {"student": { "firstname": "Andrei", "lastname": "Popescu"}, "an": 3, "grupa": "332CA", "mater
    {"student": { "firstname": "Ana", "lastname": "Georgescu"}, "an": 4, "grupa": "342C5", "materi
]

db.students.insert(studs)
```

Selectarea datelor

Select

În MongoDB operația de selecție a datelor este `find()`. Sintaxa comenzii este:

```
db.collection.find(query, projection)
```

Unde:

- `query` reprezintă lista de filtre
- `projection` reprezintă câmpurile care vor fi afișate

Dacă parametrul `query` lipsește, atunci se selectează toate documentele. Dacă parametrul `projection` lipsește atunci se afișează toate câmpurile documentului. Comanda `find()` face paginație, arătând în consolă 10 înregistrări.

Să se selecteze toți studenții din colecția students, baza de date faculty:

```
db.students.find()
```

Dacă se dorește întoarcerea unei singur document se folosește `findOne()`:

```
db.students.findOne()
```

Dacă se dorește întoarcerea unui număr `n` de documente se folosește `limit(n)`:

```
db.students.find().limit(3)
```

Dacă se dorește sortarea rezultatelor după valoare unui anumit câmp se folosește `sort({label_1: order, label_2: order,…, label_n:order})` unde `order` este `1` pentru ascendent și `-1` pentru descendent.

```
db.students.find().sort({"grupa": 1})
```

Dacă se dorește afișarea într-un mod citibil în consolă se folosește `pretty()`:

```
db.students.find().pretty()
```

`pretty`, `limit` și `sort` se pot folosi împreună:

```
db.students.find().sort({"student.firstname": 1}).limit(3).pretty()
```

Să se selecteze primele 4 rezultate ale unei cereri care întoarce toți studenți ordonați descrescător după nume:

```
db.students.find().sort({"student.firstname": -1}).limit(4);
```

Filtrarea cererilor

Pentru a filtra rezultatele se folosește parametru `query`.

Pentru comparație avem:

- `:` - egalitate
- `$ne` - diferit de (not equal)

- `$gt` - mai mare
- `$gte` - mai mare sau egal
- `$lt` - mai mic
- `$lte` - mai mic sau egal
- `$in` - căutare într-o listă
- `$all` - căutare cu egalitate pe toate elementele dintr-o listă

```
db.students.find({"student.firstname": "Ion"})
db.students.find({an: {$gte: 4}})
db.students.find({"materii.nume": {$in: ['SCAD', 'IA']} })
db.students.find({"grupa": /^341/ })
db.students.find({"cunostinte": {$all: [/^J/, /^C/ ]}})
```

Operatori logici:

- `$or` - sau logic
- `$and` - și logic
- `$not` - negare logic

```
db.students.find({$or:  [{"student.lastname": "Ionescu"}, {"cunostinte": /^C/}]})
db.students.find({$and: [{"student.lastname": "Ionescu"}, {"cunostinte": /^C/}]})
```

Operatori utili pentru vectori:

- `$size` pentru a verifica dimensiune
- `0, 1, 2,…` pentru poziționare (indexare vectorilor începe de la 0)

```
db.students.find({"cunostinte": {$size: 2}})
db.students.find({"cunostinte.2": "SQL"})
db.students.find({"materii.0.nume": "BD2"})
```

Pentru a verifica existența unui câmp se folosește `$exists`:

```
db.students.find({"sef": true})
db.students.find({"sef": {$exists: true}})
db.students.find({"sef": false})
db.students.find({"sef": {$exists: false}})
```

Să se selecteze toți studenții din anul 4 care au restanțe (au materii din ani mai mici) și știu limbajele de programare C și SQL:

```
db.students.find({an: 4, "materii.an": {$lt: 4}, "cunostinte": {$all: ["C", "SQL"]}})
```

Proiecția

Pentru a proiecta doar anumite câmpuri se folosește parametrul `projection`. În cazul în care nu se folosește filtrare se pun acolade goale pentru `query`.

```
db.students.find({}, {"student.firstname": 1})
```

O cerere cu `projection` va întoarce doar câmpurile specificate și `_id`. Dacă se dorește să se elimine câmpul `_id` atunci proiecția trebuie să conțină și `"_id": 0`:

```
db.students.find({}, {"_id": 0, "student.firstname": 1})
```

Să se selecteze studenții care nu sunt șefi și au cunoștințe de “Python” pe a doua poziție din vectorul de cunoștințe. Afișați doar numele, prenumele și vectorul de cunoștințe.

```
db.students.find(
  {
    "sef": { $exists: false },
    "cunostinte.1": "Python",
  },
  { "_id": 0, "student.firstname": 1, "student.lastname": 1, "cunostinte": 1 }
);
```

Modificarea datelor

Pentru a modifica datele se poate folosi una din comenzile:

```
db.collection.update(<query>, <update>, <options>)
```

Unde:

- `query` - filtrează datele
- `update` - setează date care trebuie modificate
- `options` - diferite opțiuni, e.g.:
- `upsert` : - crează un nou document în cazul în care nu se găsește nici un document cu filtrul
- `multi` : - modifică mai multe documente

Pentru a seta o valoare se folosește `$set` :

```
db.students.update({"student.firstname": "George"}, {$set: {"grupa": "342C1"}})
db.students.update({an: 4, "student.firstname": "Grigore"}, {$set: {cunostinte: ["C++"]}})
```

Folosind `$set` se suprascrie valoarea pentru câmp. În cazul în care dorim sa modificăm o structură de tip vector folosiți `$push` . Dacă se dorește să se adauge mai multe valori, se poate folosi $push împreună cu `each`.

```
db.students.update({an: 4, "student.firstname": "Grigore"}, {$push: {"cunostinte": "C"}})
db.students.update({an: 4, "student.firstname": "Grigore"}, {$push: {"cunostinte":{$each: ["Python
```

Dacă se dorește să se modifice valoare unui element dintr-o listă sau a unui câmp dintr-un document imbricat care se găsește în listă se poate folosi $. Trebuie să selectați și valoare/ câmpul care se dorește să fie modificat în query:

```
db.students.update({"student.firstname": "Grigore", "cunostinte": "C"}, {$set: {"cunostinte.$": "J
db.students.update({"student.firstname": "Grigore", "materii.nume": "SRIC"}, {$set: {"materii.$.nu
db.students.update({"student.firstname": "Grigore", "materii.nume": "SPG"}, {$set: {"materii.$.num
```

Să se adauge la toți studenți de anul 3 materiile EGC și LFA (nume si an) folosind o singură comandă.

```
db.students.update({an: 3}, {$push: {materii: {$each: [{nume: "EGC", an: 3}, {nume: "LFA", an: 3}]
```

Alte funcții care modifică datele sunt:

```
db.collection.updateOne(<query>, <update>, <options>)

db.collection.updateMany(<query>, <update>, <options>)
```

Ștergerea datelor

Pentru a șterge un document dintr-o colecție se folosește comanda:

```
db.collection.remove(<query>, <justOne>)
```

Unde:

- `query` - filtrează datele
- `justOne` - boolean pentru a șterge unul (true) sau mai multe (false - implicit) documente

```
 db.students.remove({"an": {$lt: 4}})
```

Pentru a șterge toate documentele dintr-o colecție se folosește:

```
db.collection.remove({})
```

```
db.students.remove({})
```

Alte comenzi care pot fi utilizare pentru a șterge documente dintr-o colecție sunt:

```
db.collection.deleteOne(<filter>)

db.collection.deleteMany(<filter>)
```

Să se șteargă doar un student care are materia BD2.

```
db.students.deleteOne({"materii.nume": "BD2"})
```

Indexare

MongoDB suport diferite tipuri de indecși. Indecși sunt utilizați pentru a crește performanțele operațiilor de căutare. Indecși sunt creați la nivel de colecție. Pentru a crea un index se folosește comanda createIndex(). La creare se specifică numele câmpului (i.e., label) și modul de ordonare a indexului (i.e., order)

Indecși simpli:

```
db.collection.createIndex({“label”: order})
```

Indecși compusi:

```
db.collection.createIndex({“label_1”: order_1, “label_2”: order_2, …})
```

Indecși pe documente imbricate:

```
db.collection.createIndex({“label_1.nested_label”: order_1, “label_2.nested_label”: order_2, …})
```

Indecși geo-spațiali:

```
db.collection.createIndex({“label_1.nested_label”: “2d”})
```

Indecși textuali (o colecție poate să aibă un singur index textual, dar acesta poate fi compus):

```
db.collection.createIndex({“label_1”: “text”, “label_2”: “text”, …})
```

Indecși hashed (utili pentru sharding):

```
db.collection.createIndex({“label”: “hashed”})
```

```
db.documents.createIndex({author: 1})
db.documents.createIndex({gender: 1, age: 1})
db.documents.createIndex({"words.word": 1})
db.documents.createIndex({"geoLocation": "2d"})
db.documents.createIndex({"lemmaText": "text"})
db.documents.createIndex({"date": "hashed"})
```

Pentru a vedea toți indecșii se folosește comanda

```
db.collection.getIndexes()
```

Pentru a șterge un index se folosește comanda

```
db.collection.dropIndexes(“index_name”)
```

Pentru a șterge toți indecșii (cu excepția indexului de pe cheia primară) se folosește comanda

```
db.collection.dropIndexes()
```

Utilizare indecși geo-spațiali

- Pentru a găsi documentele de lângă un punct `p=[x, y]` se folosește `$near`
- Pentru a găsi documentele care se găsesc în diferite figuri geometrice se folosește `$within`:
    - Pentru un cerc cu centrul într-un punct `p=[x, y]` și o rază `r` se folosește $center
    - Pentru un dreptunghi definit prin punctele din stânga jos și dreapta sus se folosește `$box`
    - Pentru un poligon dat de o listă puncte se folosește `$polygon`

```
db.documents.find( { geoLocation: { $near: [25, 25] } }, {geoLocation: 1} )
db.documents.find( { geoLocation: { $within: { $center: [[25, 10], 3] } } }, {geoLocation: 1} )
db.documents.find( { geoLocation: { $within: { $box: [[25, 40], [30, 45]] } } }, { geoLocation: 1
db.documents.find( { geoLocation: { $within: { $polygon: [[20, 40], [40, 50], [30, 30]] } } },  {
```

Utilizare indecși textuali

Pentru a utiliza un index textual se folosește următoare sintaxă:

```
{
  $text:
    {
      $search: <string>,
      $language: <string>,
      $caseSensitive: <boolean>,
      $diacriticSensitive: <boolean>
    }
}
```

Unde:

- `$search` - este șirul de cuvinte căutat sau fraza căutată
- `$language` - este limba pentru cuvintele căutate
- `$caseSensitive` - dacă se caută case sensitive sau nu
- `$diacriticSensitive` - dacă se ține cont de diacritice sau nu

```
# Caută un cuvânt
db.documents.find( { $text: { $search: "coffee", $language: "english", $caseSensitive: false, $dia

# Caută mai multe cuvinte (se folosește operatorul or)
db.documents.find( { $text: { $search: "coffee cup", $language: "english", $caseSensitive: false,

# Caută o frază
db.documents.find( { $text: { $search: "\"heaven tonight\"", $language: "english", $caseSensitive:

# Caută o frază cu un scor de relevanță
db.documents.find( { $text: { $search: "\"heaven tonight\"", $language: "english", $caseSensitive:
```

Să se găsească toate documentele care se află într-un dreptunghi dat de punctele [20, -100] și [40, 90], care să conțină cuvintele tech și engineering. Cererea să se folosească de indexul textual. Afișeți câmpul rawText. Să se refacă cererea utilizând alt câmp din document.

```
db.documents.find(
  {
    $text: {
      $search: "tech engineering",
      $language: "english",
      $caseSensitive: false,
      $diacriticSensitive: false,
    },
    geoLocation: {
      $within: {
        $box: [
          [20, -100],
          [40, 90],
        ],
      },
    },
  },
  { rawText: 1 }
);
```

Funcții

Să se scrie o funcție care primește un query pentru filtrare și împarte lemmaText în cuvinte. Funcția va întoarce un vector.

```
tokenization = function (q) {
  var cursor = db.documents.find(q, { _id: 0, lemmaText: 1 });
  var tokens = Array();
  cursor.forEach(function (elem) {
    tokens = tokens.concat(elem["lemmaText"].split(" "));
  });
  return tokens;
};

// utilizate
var q = { gender: "male" };
tokens = tokenization(q);
```

Să se scrie o funcție numită countWords care primește un query pentru filtrare și numără aparițiile unui cuvânt. Funcția va întoarce un obiect de forma {word_1: count, word_2: count, …}. Folosiți funcția tokenization.

```
countWords = function (q) {
  var tokens = tokenization(q);
  var count = {};
  tokens.forEach(function (word) {
    if (count[word]) {
      count[word] += 1;
    } else {
      count[word] = 1;
    }
  });
  return count;
};

var q1 = {
  gender: "female"
};

countWords(q1);
```

Funcții stocate

Funcțiile pot fi stocate la nivelul MongoDB. Pentru acest lucru se folosește o colecție specială

numită system.js. Stocarea se face cu comanda:

```
db.system.js.save(_id: “function_name”, value: “function_body”)
```

Pentru a lucra cu funcțiile stocare (a le încărca în sesiune) se folosește comanda:

```
db.loadServerScripts()
```

```
db.system.js.save(
   {
     _id: "tokenization",
     value : function(q){
                var cursor = db.documents.find(q,{"_id": 0, lemmaText: 1});
                var tokens = Array();
                cursor.forEach(function(elem){
                    tokens = tokens.concat(elem["lemmaText"].split(" "));
                });
                return tokens;
            }
   }
)

db.loadServerScripts()
```

Salvați și apoi încărcați în sesiune funcția countWords.

```
db.system.js.save(
   {
     _id: "countWords",
     value : function(q){
                var tokens = tokenization(q);
                var count = {};
                tokens.forEach(function(word){
                    if(count[word]){
                        count[word] += 1;
                    } else {
                        count[word] = 1;
                    }
                });
                return count;
            }
   }
)

db.loadServerScripts()
```

Agregarea datelor

Funcții de agregare

- Count

```
db.collection.count(query)

// Cate documente au lungimea textului lematizat 10
db.documents.count({lemmaTextLength: 10})
```

- Distinct

```
db.collection.distinct(field, query)

// afiseaza varstele distincte
db.documents.distinct("age")

// afiseaza varstele distincte pentru female
db.documents.distinct("age", {gender: "female"}).sort()
```

Afișați toate cuvintele distincte. Folosiți câmpul words.

```
db.documents.distinct("words.word")
```

Aggregation Pipeline

```
db.collection.aggregate(
    { $match: <query> },
    { $unwind: <array> },
    { $project: <projection>},
    { $group: <aggregation_group, aggregation_functions> },
    { $sort: <sorting_fields> },
)
```

Unde:

- `$match` - filtrează documentele de intrare după un query
- `$unwind` - procesează elementele dintr-un vector
- `$project` - proiectează câmpurile documentului
- `$group` - grupează documentele și aplică funcții de agregare
- `$sort` - reordonează documentele de intrare după o cheie

Nota: Contează ordinea în care sunt folosiți operatorii.

Să se utilizeze Aggregation Pipeline pentru a calcula numărul de apariții ale cuvintelor.

```
db.documents.aggregate([
    { $match: q },
    { $project: { words: { $split: ["$lemmaText", " "]}}},
    { $unwind: "$words" },
    { $group: { _id: "$words", counts: { $sum: 1 } } }
])
```

Să se utilizeze Aggregation Pipeline pentru a calcula numărul de apariții ale cuvintelor folosind coloana words.

```
var q = {
  $or: [
    { lemmaText: { $regex: ".*" + "a" + ".*" } },
    { lemmaText: { $regex: ".*" + "e" + ".*" } },
    { lemmaText: { $regex: ".*" + "i" + ".*" } },
    { lemmaText: { $regex: ".*" + "o" + ".*" } },
    { lemmaText: { $regex: ".*" + "u" + ".*" } }
  ]
};

db.documents.aggregate([
    { $match: q },
    { $unwind: "$words" },
    { $group: { _id: "$words.word", counts: { $sum: 1 } } }
])
```

MapReduce

```
db.collection.mapReduce(
    mapFunction,
    reduceFunction,
    {
        finalize: <function>,
        out: <output>,
        query: <document>,
        sort: <document>,
        limit: <number>,
        scope: <document>
    }
)
```

Unde:

- `mapFunction` - funcția map
- `reduceFunction` - funcția reduce
- `finalize` - (opțional) funcție pentru finalizare, se aplică după ce se termină procesul și datele sunt agregate
- `out` - (opțional) colecția unde va fi salvat rezultatul
- `query` - (opțional) filtru pentru documentele de intrare
- `sort` - (opțional) ordonează documentele de intrare după cheie
- `limit` - (opțional) limitează numărul de documente de intrare
- `scope` - (opțional) variabile din exterior care pot fi folosite în funcțiile map, reduce și finalize

Comanda mapReduce este o mapare peste comanda mapReduce din MongoDB

```
db.runCommand(
    {
        mapReduce: <collection>,
        map: <function>,
        reduce: <function>,
        finalize: <function>,
        out: <output>,
        query: <document>,
        sort: <document>,
        limit: <number>,
        scope: <document>
    }
)
```

Să se utilizeze MapReduce pentru a calcula numărul de apariții ale cuvintelor.

```
mapFunction = function() {
    var tokens = this.lemmaText.split(" ");
    for (var idx=0; idx<tokens.length; idx++){
        emit(tokens[idx], 1);
    }
}

reduceFunction = function(key, values) {
    return Array.sum(values);
};

var q = {"gender": "female"}
db.documents.mapReduce(mapFunction, reduceFunction, {query: q, out: "wordCounts"});

db.wordCounts.count()

db.wordCounts.find({"_id": "0"})

db.wordCounts.drop()
```

Să se utilizeze MapReduce pentru a calcula numărul de apariții ale cuvintelor folosind câmpul words.

```
mapFunction = function() {
    for (var idx=0; idx<this.words.length; idx++){
        emit(this.words[idx].word, 1);
    }
}

reduceFunction = function(key, values) {
    return Array.sum(values);
};

var q = {
  $or: [
    { lemmaText: { $regex: ".*" + "a" + ".*" } },
    { lemmaText: { $regex: ".*" + "e" + ".*" } },
    { lemmaText: { $regex: ".*" + "i" + ".*" } },
    { lemmaText: { $regex: ".*" + "o" + ".*" } },
    { lemmaText: { $regex: ".*" + "u" + ".*" } }
  ]
};

db.documents.mapReduce(mapFunction, reduceFunction, {query: q, out: "wordCounts"});

db.wordCounts.count()

db.wordCounts.find({"_id": "0"})

db.wordCounts.drop()
```

Exerciții

Să se găsească documentele care conțin cel mai popular hashtag.

```
const mostPopularHashtag = db.documents.aggregate([
  { $unwind: "$hashtags" },
  { $group: { _id: "$hashtags", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 1 }
]).next();

printjson(mostPopularHashtag);

db.documents.find({ hashtags: mostPopularHashtag._id });
```

Să se găsească cel mai activ autor după numărul de documente create.

```
const mostActiveAuthor = db.documents.aggregate([
  { $group: { _id: "$author", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 1 }
]).next();

printjson(mostActiveAuthor);

db.documents.find({ author: mostActiveAuthor._id });
```

Pentru toate documentele care content cel mai popular hashtag vedeti distributia de varsta.

```
const mostPopularHashtag = db.documents.aggregate([
  { $unwind: "$hashtags" },
  { $group: { _id: "$hashtags", count: { $sum: 1 } } },
  { $sort: { count: -1 } },
  { $limit: 1 }
]).next();

printjson(mostPopularHashtag);

const ageDistribution = db.documents.aggregate([
  { $match: { hashtags: mostPopularHashtag._id } },
  { $group: { _id: "$age", count: { $sum: 1 } } }
]).toArray();

printjson(ageDistribution);
```

Cate documente cu autori Tineri (<= 35 de ani) avem pentru fiecare gen?

```
const youngAuthors = db.documents.aggregate([
  { $match: { age: { $lte: 35 } } },
  { $group: { _id: "$gender", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
]).toArray();

printjson(youngAuthors);
```

Pentru documentele care se afla in cercul de centru (20,20) si raza 100, care sunt top 3 cele mai folosite Tipuri de Enitati (campul namedEntities.type; la egalitate sortati alfabetic)?

```
const top3Entities = db.documents.aggregate([
  {
    $match: {
      geoLocation: {
        $geoWithin: {
          $center: [[20, 20], 100]
        }
      }
    }
  },
  { $unwind: "$namedEntities" },
  { $group: { _id: "$namedEntities.type", count: { $sum: 1 } } },
  { $sort: { count: -1, _id: 1 } },
  { $limit: 3 }
]).toArray();

printjson(top3Entities);
```